



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ
ФИЛИАЛ ПЛОВДИВ

ФАКУЛТЕТ ПО ЕЛЕКТРОНИКА И АВТОМАТИКА
КАТЕДРА “КОМПЮТЪРНИ СИСТЕМИ”

ПРОЕКТИРАНЕ НА СВРЪХГОЛЕМИ ИНТЕГРАЛНИ
СХЕМИ

Протокол № 3

Изготвил: Любомир Ламбрев
Група: 141
Факултетен номер: 614973

Регистър (Registers)

1. Теоретична част

Регистърът е цифров логически елемент, който служи за съхранение на двоични данни. Той се състои от D-тригерни клетки, които запазват стойността на входния сигнал при наличие на тактов сигнал (**CLK**).

Регистрите могат да бъдат използвани за временни запамятаващи елементи, буферизиране на данни и синхронизация в цифрови системи.

Основни характеристики на регистъра:

- Размер (n-битов) – определя броя на битовете, които могат да се съхраняват.
- Тактов сигнал (CLK) – определя момента на актуализация на стойностите.
- Синхронно зареждане (LOAD) – активира процеса на запис на данни в регистъра.
- Анулиране (RST) – нулира съдържанието на регистъра.

Регистърът, реализиран в този протокол, има следните входове и изходи:

- I (Input) – входен сигнал, който трябва да бъде записан.
- CLK – тактов сигнал, който определя момента на записа.
- LOAD – сигнал, който разрешава записването на входните данни в регистъра.
- RST – сигнал за нулиране на съдържанието.
- Q (Output) – текущото съдържание на регистъра.

Очаквани резултати

RST	LOAD	CLK	I	Q	описание
0	X	X	XX	00	Регистърът се нулира
1	1	0	10	00	Готов за запис
1	1	1	10	10	Данните се записват
1	0	1	11	10	Q не се променя (LOAD = 0)
1	1	1	01	01	Записана нова стойност

2. Описание на VHDL код

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.std_logic_unsigned.all;
```

```
entity REG is
```

```
    generic (n : natural := 2);
```

```
    port (
```

```
        I : in std_logic_vector(n-1 downto 0);
```

```
        CLK : in std_logic;
```

```
        LOAD : in std_logic;
```

```
        RST : in std_logic;
```

```
        Q : out std_logic_vector(n-1 downto 0)
```

```
    );
```

```
end REG;
```

```
architecture beh of REG is
```

```
    signal Q_tmp : std_logic_vector(n-1 downto 0);
```

```
begin
```

```
    process (CLK, RST)
```

```

begin
    if RST = '0' then
        Q_tmp <= (others => '0');
    elsif (CLK'event and CLK = '1') then
        if LOAD = '1' then
            Q_tmp <= I;
        end if;
    end if;
end process;

```

```

Q <= Q_tmp;

```

```

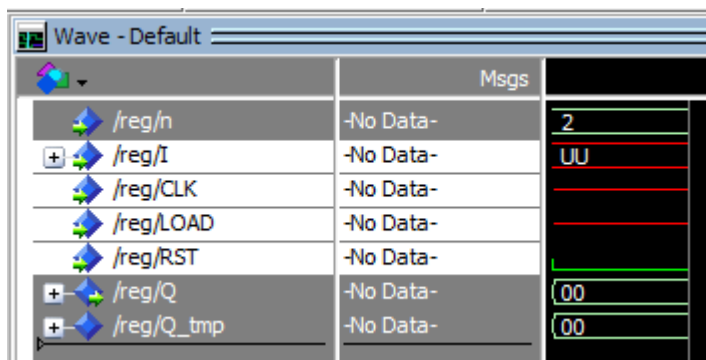
end beh;

```

3. Симулиране и времедиаграми

Симулацията генерира следните резултати, които потвърждават коректна работа на Регистъра според очакваните изходи.

- (RST = 0, LOAD = X, CLK = X, I = XX) → Q = 00 (Регистърът се нулира)



- (RST = 1, LOAD = 1, CLK = 0, I = 10) → Q = 00 (Готов за запис)

Wave - Default			
	Msgs		
/reg/n	-No Data-	2	
+ /reg/I	-No Data-	UU	10
/reg/CLK	-No Data-		
/reg/LOAD	-No Data-		
/reg/RST	-No Data-		
+ /reg/Q	-No Data-	00	
+ /reg/Q_tmp	-No Data-	00	

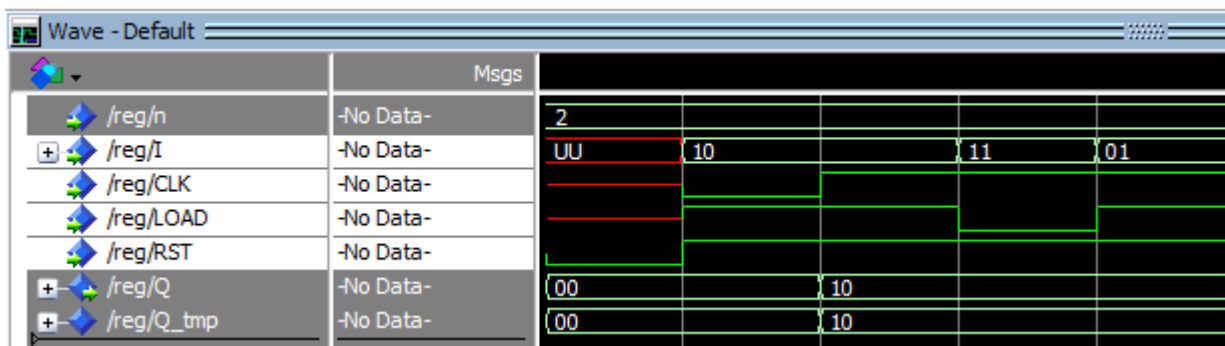
- (RST = 1, LOAD = 1, CLK = 1, I = 10) → Q = 10 (Данните се записват)

Wave - Default			
	Msgs		
/reg/n	-No Data-	2	
+ /reg/I	-No Data-	UU	10
/reg/CLK	-No Data-		
/reg/LOAD	-No Data-		
/reg/RST	-No Data-		
+ /reg/Q	-No Data-	00	10
+ /reg/Q_tmp	-No Data-	00	10

- (RST = 1, LOAD = 0, CLK = 1, I = 11) → Q = 10 (Q не се променя (LOAD = 0))

Wave - Default			
	Msgs		
/reg/n	-No Data-	2	
+ /reg/I	-No Data-	UU	10 11
/reg/CLK	-No Data-		
/reg/LOAD	-No Data-		
/reg/RST	-No Data-		
+ /reg/Q	-No Data-	00	10
+ /reg/Q_tmp	-No Data-	00	10

- (RST = 1, LOAD = 1, CLK = 1, I = 01) → Q = 01 (Записана нова стойност)



Структурно описание (twocomp)

1. Теоретична част

В този пример е реализирана логическа схема, изградена от два основни логически компонента – NAND и OR. Схемата има за цел да демонстрира как се изгражда по-сложна логика чрез свързване на базови логически елементи.

Операцията NAND (НЕ-И) връща резултат '0' само ако и двата входа са '1'. Във всички останали случаи резултатът е '1'. Тоест: $A \text{ NAND } B = \text{NOT } (A \text{ AND } B)$. Това е универсален логически елемент, чрез който може да се реализира всяка друга логическа операция.

Операцията OR (ИЛИ) връща '1', ако поне един от входовете е '1'. Само при всички входове на логически '0' изходният резултат е '0'.

Системата се състои от две входни стойности (in1, in2), които се подават към NAND елемент. Изходът от него се комбинира с трети вход (in3) чрез OR логика. Изходът out1 се влияе от резултатите на тези две операции.

in1	in2	in3	out3	out1
0	0	0	1	1
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	0	1

2. Описание на VHDL код

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity twocomp is
```

```
    port (in1, in2, in3: in std_logic;
```

```
          out1: out std_logic);
```

```
end twocomp;
```

```
architecture structure of twocomp is
```

```
    component nand1
```

```
        port(in11, in22: in std_logic;
```

```
              out0: out std_logic);
```

```
    end component;
```

```
    component xor2
```

```
        port(in33, in44: in std_logic;
```

```
              out2: out std_logic);
```

```
    end component;
```

```
signal out3: std_logic;
```

```
begin
```

```
    a1: nand1 port map (in11 => in1, in22 => in2, out0 => out3);
```

```
    a2: xor2 port map (in33 => out3, in44 => in3, out2 => out1);
```

```
end structure;
```

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity nand1 is
```

```
    port (in11, in22: in std_logic;
```

```
          out0: out std_logic);
```

```
end nand1;
```

```
architecture dataflow1 of nand1 is
```

```
begin
```

```
    out0 <= in11 nand in22;
```

```
end dataflow1;
```

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity xor2 is
```

```
    port (in33, in44: in std_logic;
```

```
          out2: out std_logic);
```

```
end xor2;
```

```
architecture dataflow2 of xor2 is
```

```
begin
```

```
    out2 <= in33 or in44;
```

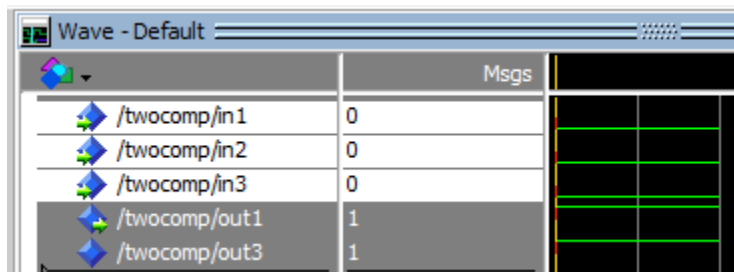
```
end dataflow2;
```

3. Симулиране и времедиаграми

Симулацията на описания VHDL модул се извършва чрез прилагане на входни сигнали in1, in2, in3 и наблюдение на изходния сигнал out0 и out1. Симулацията генерира следните резултати, които потвърждават коректна работа според очакваните изходи.

Очаквани резултати:

- (in1 = 0, in2 = 0, in3 = 0) → 1 (out3), 1 (out1)



- (in1 = 0, in2 = 0, in3 = 1) → 1 (out3), 1 (out1)

Wave - Default		
	Msgs	
/twocomp/in 1	0	
/twocomp/in 2	0	
/twocomp/in 3	1	
/twocomp/out 1	1	
/twocomp/out 3	1	

- $(in1 = 0, in2 = 1, in3 = 0) \rightarrow 1 (out3), 1 (out1)$

Wave - Default		
	Msgs	
/twocomp/in 1	0	
/twocomp/in 2	1	
/twocomp/in 3	0	
/twocomp/out 1	1	
/twocomp/out 3	1	

- $(in1 = 0, in2 = 1, in3 = 1) \rightarrow 1 (out3), 1 (out1)$

Wave - Default		
	Msgs	
/twocomp/in 1	0	
/twocomp/in 2	1	
/twocomp/in 3	1	
/twocomp/out 1	1	
/twocomp/out 3	1	

- $(in1 = 1, in2 = 0, in3 = 0) \rightarrow 1 (out3), 1 (out1)$

Wave - Default		
	Msgs	
/twocomp/in 1	1	
/twocomp/in 2	0	
/twocomp/in 3	0	
/twocomp/out 1	1	
/twocomp/out 3	1	

- $(in1 = 1, in2 = 0, in3 = 1) \rightarrow 1 (out3), 1 (out1)$

Wave - Default		
	Msgs	
/twocomp/in1	1	
/twocomp/in2	0	
/twocomp/in3	1	
/twocomp/out1	1	
/twocomp/out3	1	

- (in1 = 1, in2 = 1, in3 = 0) → 0 (out3), 0 (out1)

Wave - Default		
	Msgs	
/twocomp/in1	1	
/twocomp/in2	1	
/twocomp/in3	0	
/twocomp/out1	0	
/twocomp/out3	0	

- (in1 = 1, in2 = 1, in3 = 1) → 0 (out3), 1 (out1)

Wave - Default		
	Msgs	
/twocomp/in1	1	
/twocomp/in2	1	
/twocomp/in3	1	
/twocomp/out1	1	
/twocomp/out3	0	